SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service: SNR Sons Charitable Trust]
[Autonomous Institution, Reaccredited by NAAC with 'A+' Grade]
[Approved by AICTE and Permanently Affiliated to Anna University, Chennai]
[ISO 9001-2015 Certified and all eligible programmes Accredited by NBA]
VATTAMALAIPALAYAM, N.G.G.O. COLONY POST,
COIMBATORE – 641 022

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

20CS259 – FOUNDATIONS OF BLOCK CHAIN TECHNOLOGY

LAB RECORD

ACADEMIC YEAR: 2024-2025

BATCH: 2022-2026

MAY 2025

SRI RAMAKRISHNA ENGINEERING COLLEGE

[Educational Service: SNR Sons Charitable Trust]
[Autonomous Institution, Reaccredited by NAAC with 'A+' Grade]
[Approved by AICTE and Permanently Affiliated to Anna University, Chennai]
[ISO 9001:2015 Certified and all eligible programmes Accredited by NBA]
Vattamalaipalayam, N.G.G.O. Colony Post, Coimbatore – 641 022.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CLASS: **III B.E.(CSE)** SEMESTER: **VI**

Certified that this is the bonafide record of work done by Mr./Ms. SAHANA.S the **20CS259 FOUNDATIONS OF BLOCK CHAIN TECHNOLOGY** of this institution for VI Semester during the academic year 2024 - 2025.

Course Coordinator **HOD – CSE**

29/4/25

Mr. Ismankhan Y M, AP/CSE

REGISTER NUMBER

7181220120 8

Submitted for the **VI semester** B.E Practical Examination held on _____ 2024-2025.

Internal Examiner **Subject Expert**

INDEX

Ex. No.	DATE	NAME OF THE EXPERIMENT	PAGE No.	MARKS	SIGNATURE
1.	23/01/25	Analyzing Bitcoin Data from Coin Market cap	01	10	
2	06/02/25	Creating and Installing a Meta Mask Wallet	04	10	
3	13/02/25	Creating a Meta Mask Wallet and linking with Ganache	08	10	
4	20/02/2025	HelloWorld Using Saw tooth	11	10	
5	08/03/2025	Certificate Generation using Dapps	14	10	
6	13/03/2025	Clinical Trial using Dapps	19	10	
7	20/03/2025	Land Registration using Etherum	23	10	
8	27/03/2025	A block chain based smart- Toll management System	25	10	
9	03/04/2025	Connecting metamask, ganache and etherum	27	10	
		Average		10	

EX.NO: 01	ANALYZING BITCOIN DATA FROM COIN MARKETCAP
DATE:23.01.25	

AIM:

To analysing and understand the various features, settings, and data provided for Bitcoin on the CoinMarketCap website.

DESCRIPTION:

Bitcoin Data from CoinMarketCap refers to the real-time and historical information provided by the CoinMarketCap website about Bitcoin (BTC), the world's first and most widely recognized cryptocurrency. CoinMarketCap is a popular platform that tracks prices, market capitalizations, volumes, and other important statistics of cryptocurrencies.

PROCEDURE:

1. Open a web browser and go to the official CoinMarketCap website at https://coinmarketcap.com/.
This platform provides real-time data and analytics for cryptocurrencies.

2. Use the search bar at the top of the page to find "Bitcoin (BTC)". Click on the result to open the Bitcoin details page.

3. On the Bitcoin page, record the current price in USD. Also, note the market capitalization and 24-hour trading volume listed on the page.

4. Optionally, you can also record other data like circulating supply, total supply, and the 24-hour price change.These help give a more complete picture of Bitcoin's current status.

5. Open Microsoft Excel or Google Sheets to begin analyzing the data. Input the collected data in a structured table format for easy calculation and comparison.

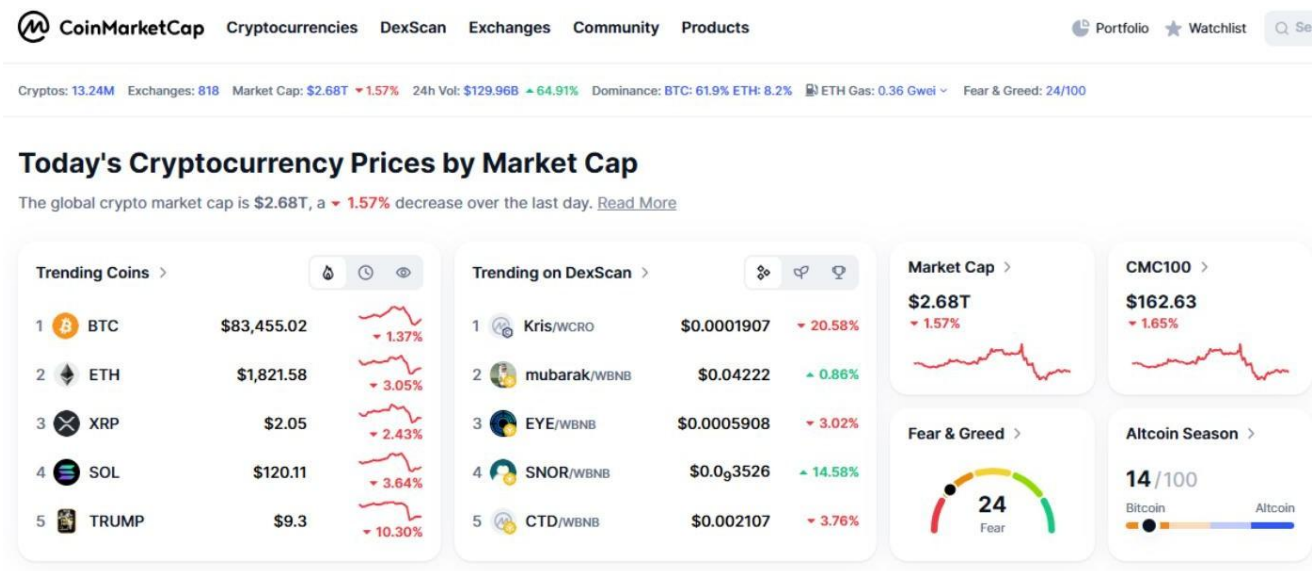
6. Calculate percentage changes in price, market cap, and trading volume over a period (e.g., 24 hours or 7 days).Use basic Excel formulas to compute the percentage increase or decrease.

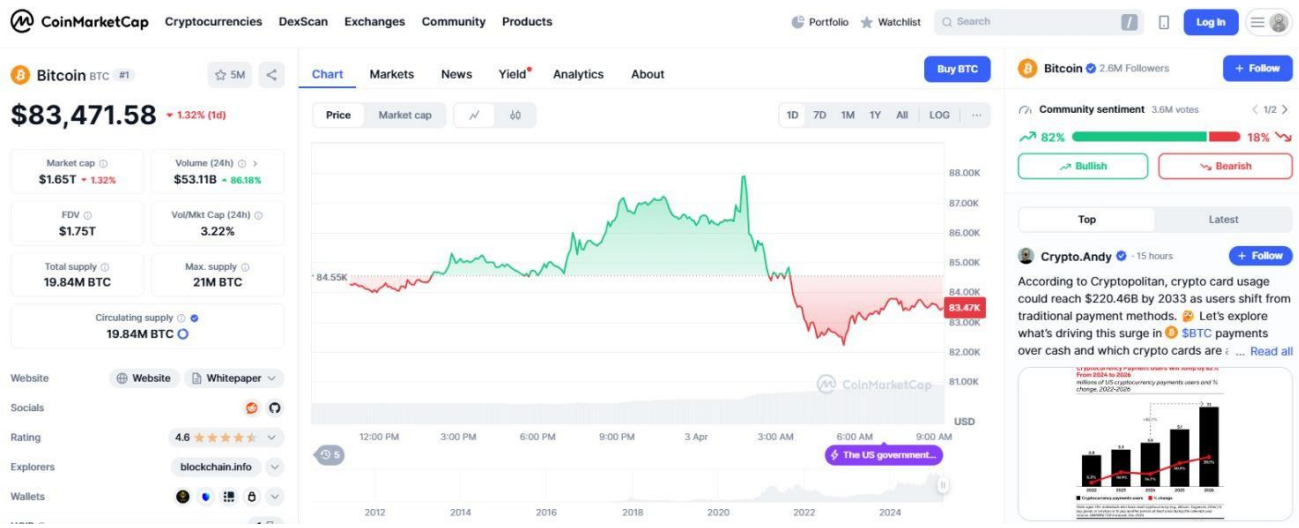
7. Create visual representations such as bar graphs or line charts in Excel or Sheets. These charts will help visualize price trends and trading volume fluctuations over time.

8. Analyze the trends in the data and identify any patterns. Discuss possible factors influencing Bitcoin's price, such as global news, economic updates, or investor sentiment.

9. Compare Bitcoin's performance with other cryptocurrencies like Ethereum or traditional assets like gold or stocks.

OUTPUT:





Compare with | TradingView

Bitcoin Markets

ALL | CEX | DEX | Spot | Perpetual | Futures | All pairs

#	Exchange	Pair	Price	+2% Depth	-2% Depth	Volume (24h)	Volume %	Confide
1	Binance	BTC/USDT	$83,557.78	$22,013,153	$24,726,592	$3,681,547,599	6.98%	
2	Binance	BTC/FDUSD	$83,552.46	$4,016,428	$6,269,193	$2,780,893,389	5.27%	
3	Bybit	BTC/USDT	$83,474.24	$14,601,279	$15,392,138	$1,355,872,009	2.57%	
4	Coinbase Exchange	BTC/USD	$83,531.57	$16,841,032	$21,934,077	$1,456,108,762	2.76%	
5	Upbit	BTC/KRW	$84,594.52	$51,763	$50,002	$254,111,906	0.48%	
6	OKX	BTC/USDT	$83,540.26	$4,704,287	$4,305,668	$1,296,960,077	2.46%	
7	Bitget	BTC/USDT	$83,499.58	$9,046,771	$1,894,043	$1,200,930,667	2.28%	
8	MEXC	BTC/USDT	$83,443.41	$8,117,519	$4,145,217	$191,001,878	0.36%	
9	Gate.io	BTC/USDT	$83,551.85	$6,750,656	$9,477,273	$912,785,279	1.73%	
10	KuCoin	BTC/USDT	$83,539.56	$5,012,969	$6,780,318	$359,896,221	0.68%	

RESULT:

Thus, successfully explored and analyze the CoinMarketCap website, understood Bitcoin's market data, historical trends, and various analytical tools and features available for user interaction and research.

EX.NO: 02	CREATING AND INSTALLING A METAMASK WALLET
DATE:06.02.25	

AIM:

To understand and perform the process of installing the MetaMask browser extension and successfully creating a secure Ethereum wallet with a backup seed phrase.

DESCRIPTION:

This exercise aims to familiarize students with the process of creating a MetaMask wallet, a popular Ethereum wallet browser extension. By completing this exercise, students will understand how to set up a MetaMask wallet, manage their accounts, and interact with decentralized applications (DApps) on the Ethereum blockchain.

PROCEDURE:

1. **Install MetaMask:**

   - Open a web browser and visit Meta Mask.
   - Install the Meta Mask browser extension by following the instructions on the website.
   - After installation, click on the Meta Mask extension icon in the browser toolbar to launch Meta Mask.

2. **Create a New Wallet:**

   - Click on the "Get Started" button in Meta Mask.
   - Follow the on-screen instructions to create a new wallet.
   - Set a strong password and click "Create" to proceed.

3. **Backup Your Seed Phrase:**

   - Meta Mask will display a **12-word seed phrase (recovery phrase)** after wallet creation.
   - Write down the seed phrase on a piece of paper and store it securely.
   - This seed phrase is essential for recovering access to your wallet if needed.

4. **Confirm Your Seed Phrase:**

   - Meta Mask will ask you to confirm your seed phrase.
   - Select the words in the correct order and click "Confirm" to proceed.

5. **Access Your Wallet:**

- After confirming the seed phrase, Meta Mask will prompt you to unlock your wallet.
- Enter your password to unlock your wallet and access your Ethereum accounts.

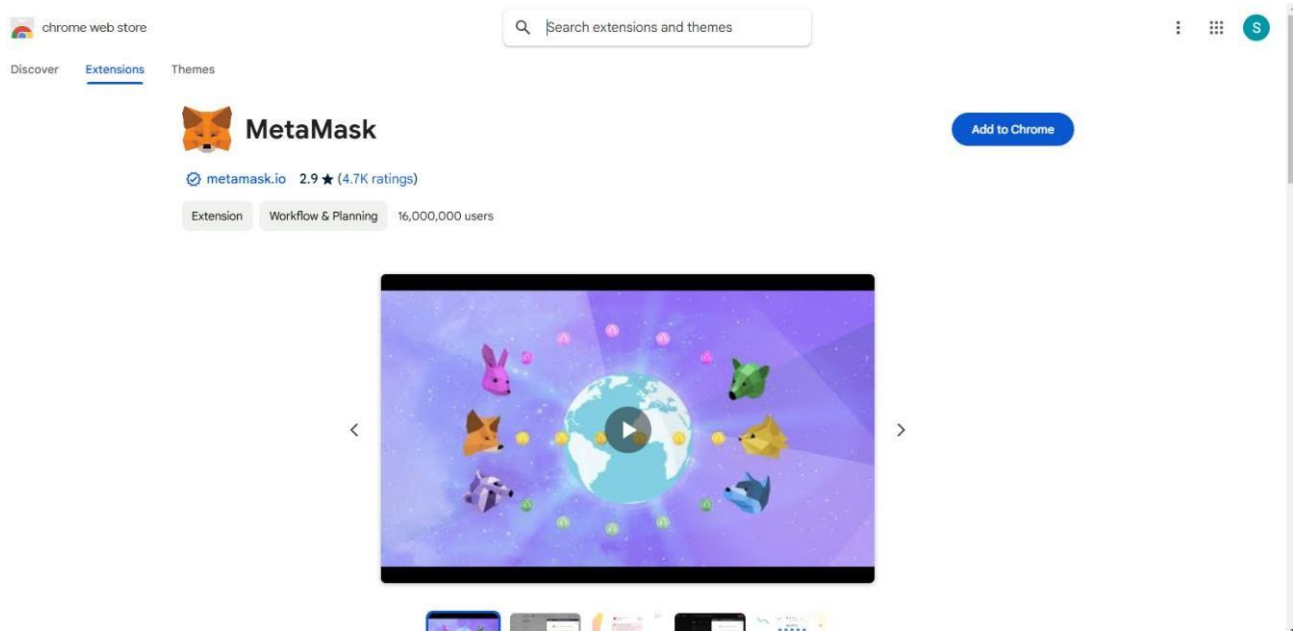
6. **Explore MetaMask Features:**

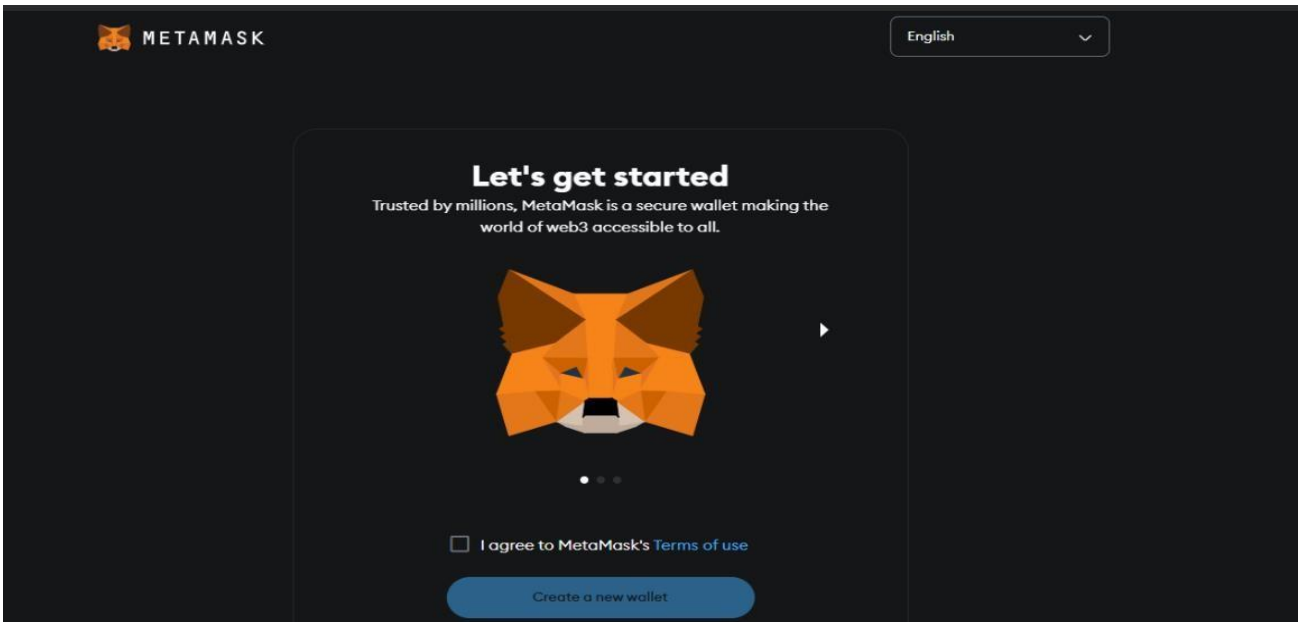
- Navigate through Meta Mask's features such as **account management, transaction history, and network settings**.
- Familiarize yourself with Ethereum **mainnet** and test networks like **Ropsten, Rinkeby, and Kovan**.

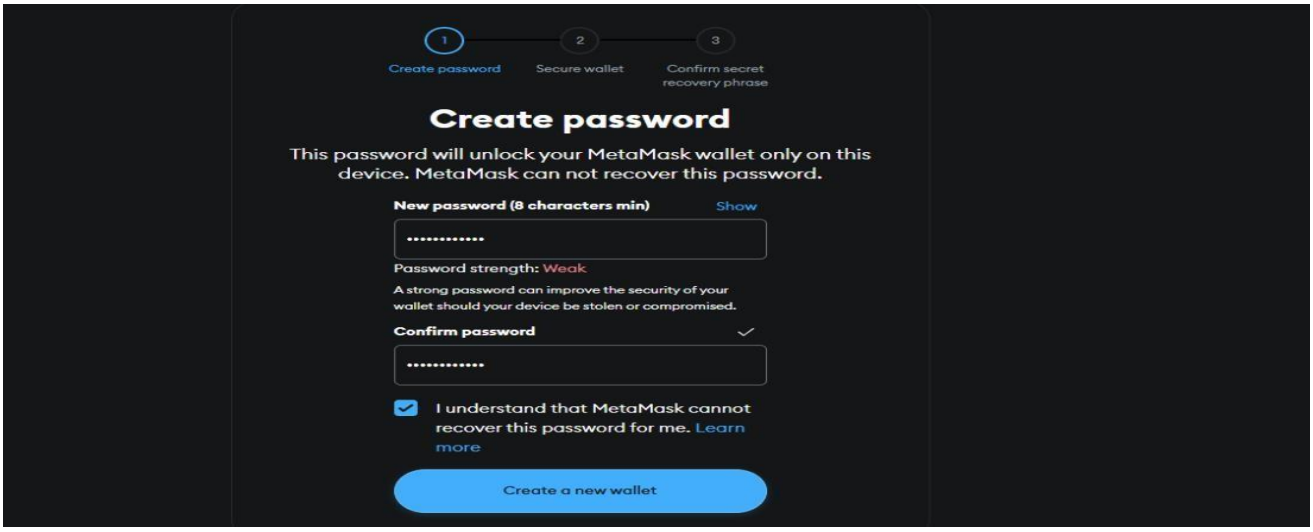
7. **Interact with DApps (Optional):**

- Visit decentralized applications (DApps) that support MetaMask integration.
- Connect your MetaMask wallet to DApps like **decentralized exchanges (DEXs), blockchain games, or DeFi platforms**.
- Perform transactions and interact with smart contracts on the blockchain.

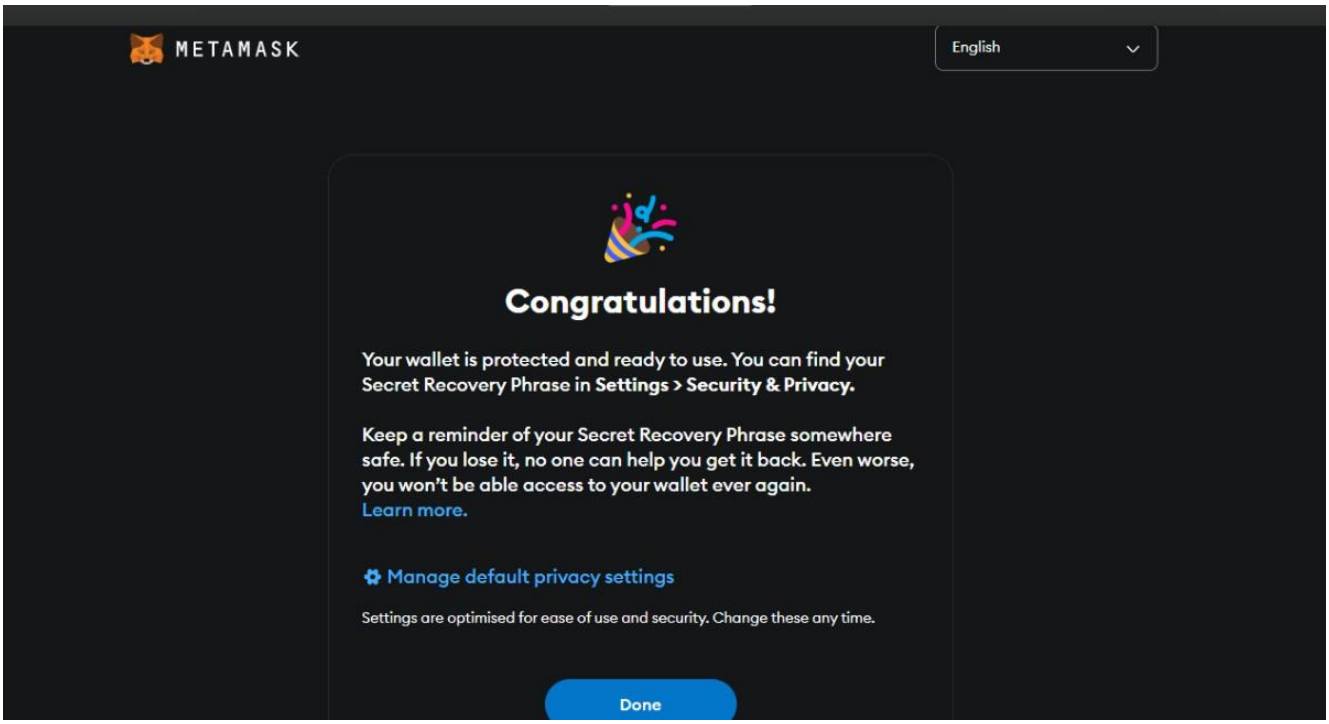
OUTPUT:

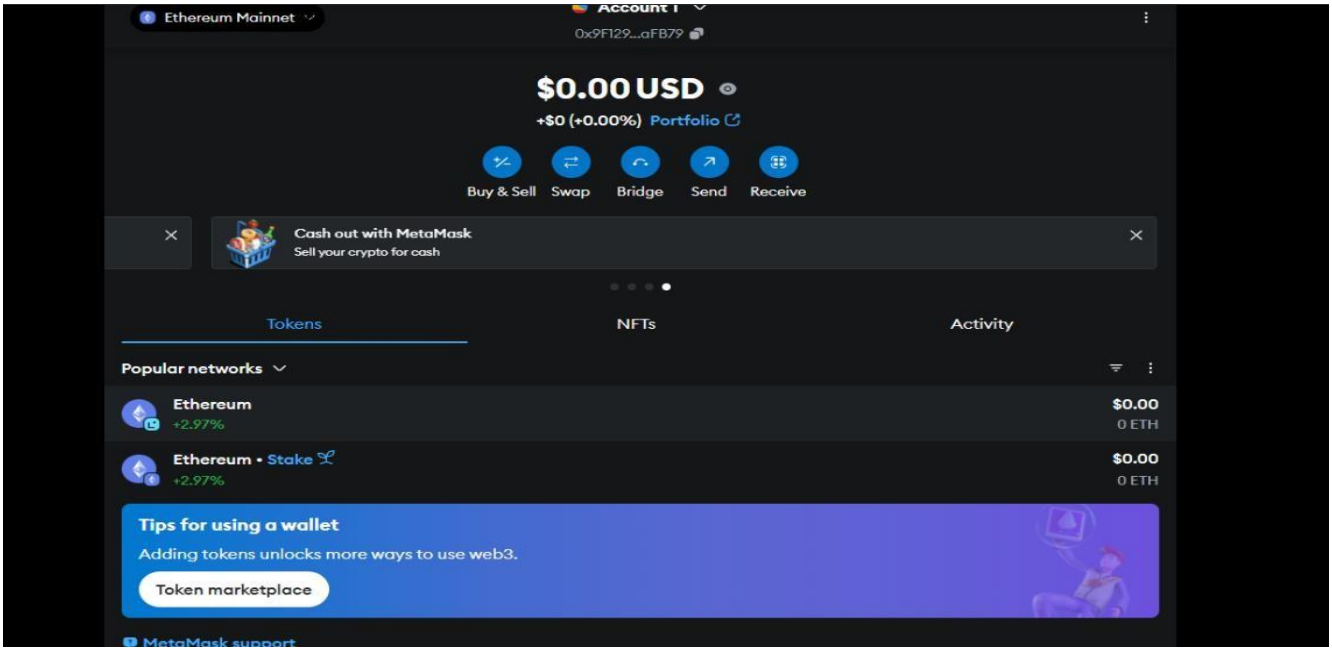



METAMASK
English
Let's get started
Trusted by millions, MetaMask is a secure wallet making the world of web3 accessible to all.
I agree to MetaMask's Terms of use
Create a new wallet


1
2
3
Create password
Secure wallet
Confirm secret recovery phrase
Create password
This password will unlock your MetaMask wallet only on this device. MetaMask can not recover this password.
New password (8 characters min)
Show
Password strength: Weak
A strong password can improve the security of your wallet should your device be stolen or compromised.
Confirm password
I understand that MetaMask cannot recover this password for me. Learn more
Create a new wallet

1
2
3
Create password
Secure wallet
Confirm secret recovery phrase
Write down your Secret Recovery Phrase
Write down this 12-word Secret Recovery Phrase and save it in a place that you trust and only you can access.
Tips:
• Write down and store in multiple secret places
• Store in a safe deposit box
1. process
2. ladder
3. actress
4. balcony
5. mandate
6. defy
7. six
8. skill
9. kingdom
10. blind
11. ginger
12. assault
Hide seed phrase
Copy to clipboard





RESULT:

Thus, successfully installed the MetaMask extension, created a new Ethereum wallet, and securely backed up the recovery seed phrase for future access and wallet recovery.

EX.NO: 03	CREATING A METAMASK WALLET AND LINKING WITH GANACHE
DATE:13.02.25	

AIM:

To create a MetaMask wallet and connect it with Ganache, a local blockchain for Ethereum development. This setup allows for secure and efficient testing of smart contracts and blockchain interactions.

DESCRIPTION:

This experiment involves setting up a MetaMask wallet and connecting it to Ganache, a local Ethereum blockchain used for development and testing. MetaMask allows users to manage Ethereum accounts and interact with DApps directly from the browser. Ganache provides a personal blockchain with pre-funded test accounts for smart contract deployment. Linking MetaMask with Ganache enables developers to simulate transactions in a secure test environment.

PROCEDURE:

Step 1: Open your preferred web browser and navigate to the official MetaMask website. Click on the "Download" button to install the MetaMask extension, and follow the on-screen instructions to complete the installation. Once installed, the MetaMask icon will appear in your browser's toolbar.

Step 2: Click on the MetaMask icon to open the extension. Follow the prompts to create a new wallet. Make sure to write down and securely store the 12-word mnemonic (seed) phrase provided. Then, set up a strong password for your wallet. Your MetaMask wallet is now successfully created and ready for use.

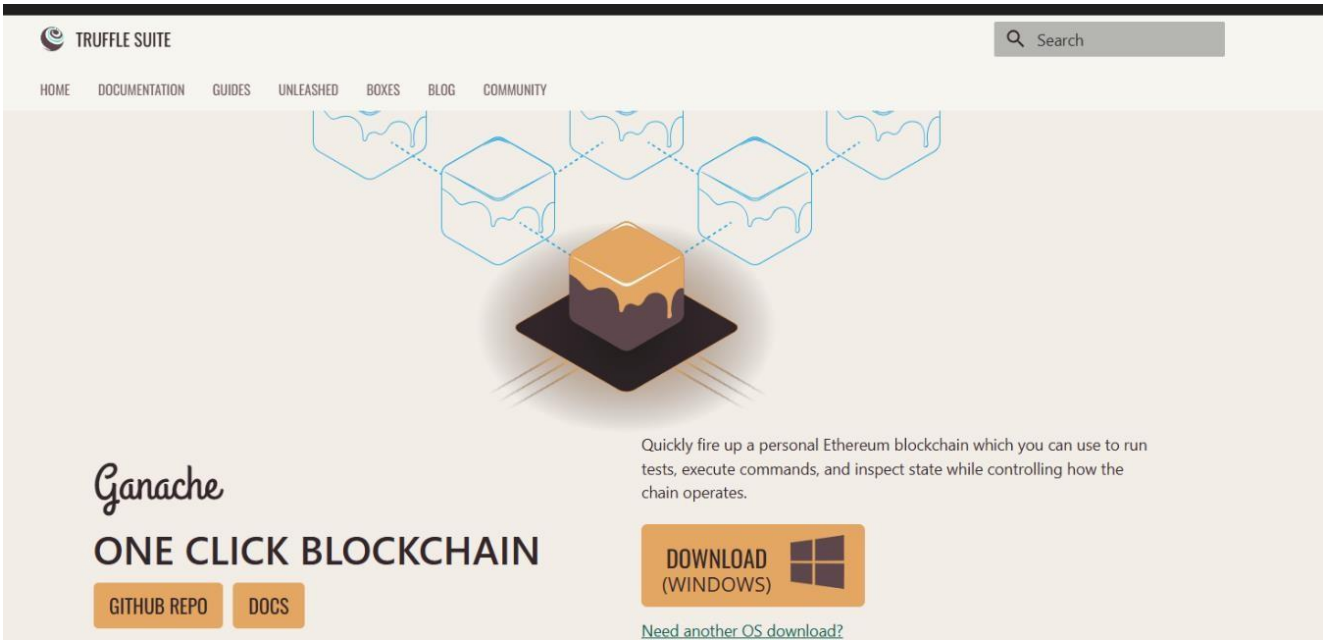
Step 3: Visit the official Ganache website and download the Ganache application suitable for your operating system. After downloading, install and launch the Ganache application on your computer.

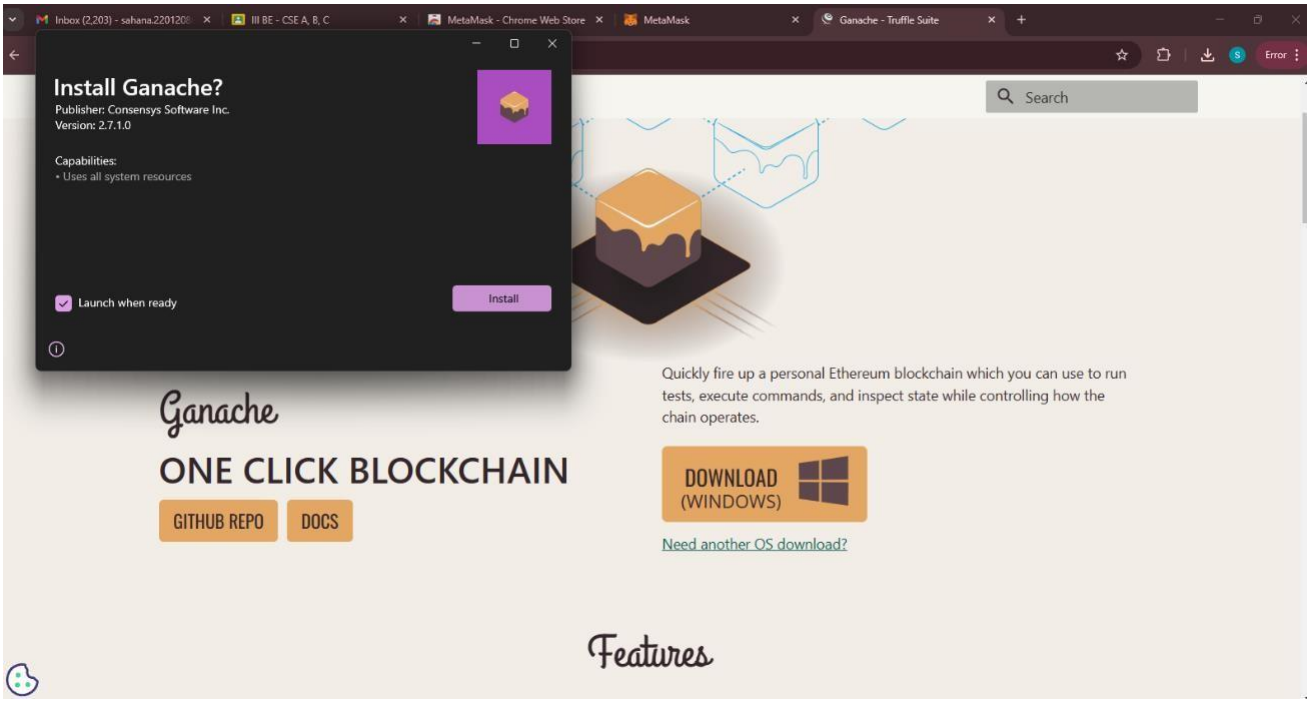
Step 4: In the Ganache interface, locate the RPC server endpoint shown at the top of the window. This is the URL you'll need to connect MetaMask to the local blockchain.

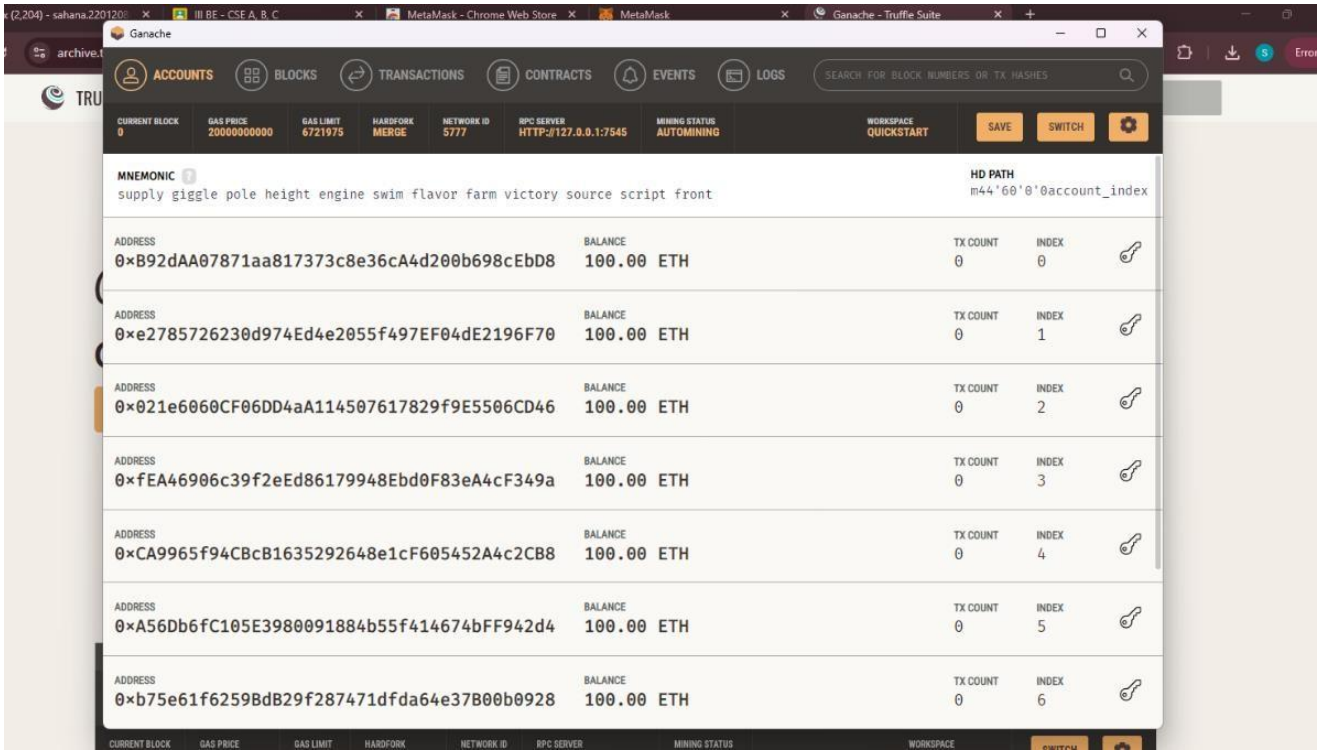
Step 5: Go back to MetaMask and click on the network dropdown menu. Choose "Add Network" or "Custom RPC" (depending on your MetaMask version). Paste the copied Ganache RPC server endpoint into the "New RPC URL" field and save the changes to add the network.

Step 6: After saving, MetaMask will be connected to the Ganache local blockchain. You should now see the pre-funded Ganache accounts in MetaMask. You can use MetaMask to deploy and test smart contracts locally using the Ganache environment.

OUTPUT:







RESULT:

Thus, MetaMask was successfully installed and configured, and a new wallet was created. It was then successfully connected to Ganache, enabling secure local blockchain testing.

EX.NO: 04	HELLO WORLD USING SAWTOOTH
DATE:20.02.25	

AIM:

To create a basic "Hello World" application using the Sawtooth blockchain platform and demonstrate deploying and interacting with a simple smart contract or transaction processor.

DESCRIPTION:

This experiment demonstrates how to create and deploy a basic HelloWorld application using the Hyperledger Sawtooth blockchain platform. The aim is to build and test a simple transaction processor that handles and logs "Hello, World!" transactions. It introduces key blockchain development concepts such as transaction families, handlers, and processor setup. The experiment helps learners understand the Sawtooth framework, local blockchain deployment, and smart contract interactions in a secure development environment.

MATERIALS REQUIRED:

- Computer with internet access
- Sawtooth Software Development Kit (SDK)
- Docker (optional, for containerized deployment)
- Text editor or Integrated Development Environment (IDE)
- Terminal or Command Prompt

PROCEDURE:

Step 1: Begin by installing the Hyperledger Sawtooth SDK on your system using the official Sawtooth documentation.If you prefer container-based deployment, install Docker and configure a local Sawtooth network using Docker Compose.

Step 2: Open a text editor or IDE and create a JavaScript file for your HelloWorld transaction processor.Write the logic to process and log "Hello, World!" transactions using the Sawtooth SDK.

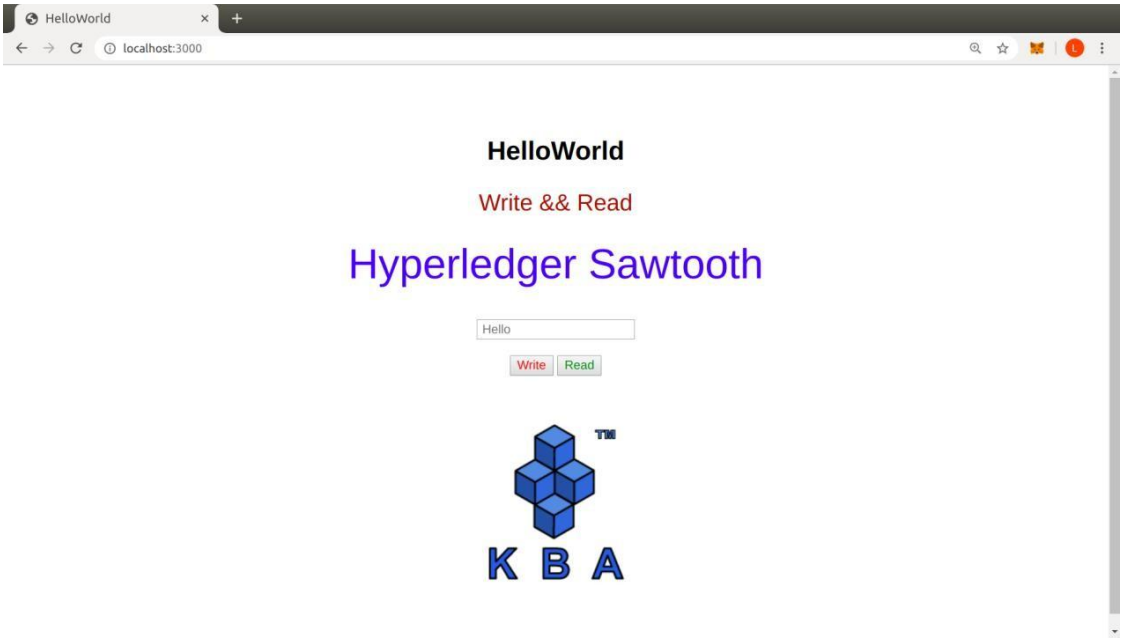
Step 3: Package your application and deploy it to the Sawtooth network using either the command-line tools or Docker.Ensure that the Sawtooth validator and required components are running before deploying the application.

Step 4: Once deployed, use the SDK or Sawtooth CLI commands to send test transactions with a "Hello, World!" payload.

CODE:

```
const { TransactionProcessor } = require('sawtooth-sdk/processor');

// Define the address prefix for the transaction family
const FAMILY_NAME = 'hello_world';
const FAMILY_VERSION = '1.0';
const PREFIX =
require('crypto').createHash('sha512').update(FAMILY_NAME).digest('hex').slice(0, 6);

// Define the transaction handler function
const handler = (transaction, context) => {
  console.log(`Received transaction: ${transaction.header.signerPublicKey}`);

  // Extract the payload from the transaction
  const payload = transaction.payload.toString();
  console.log(`Payload: ${payload}`);

  // Perform any desired processing here

  // Acknowledge successful processing
  return Promise.resolve().then(() => {
    console.log("Transaction successfully processed.");
  });
};

// Create a new Transaction Processor instance
const processor = new TransactionProcessor('tcp://localhost:4004');

// Set the transaction handler function
processor.addHandler(handler, [PREFIX]);

// Start the transaction processor
processor.start();

console.log(`Transaction processor for family ${FAMILY_NAME} version
${FAMILY_VERSION} started.`);
```

OUTPUT:



RESULT:

Thus, a HelloWorld application was successfully created and deployed on the sawtooth blockchain. The experiment demonstrated processing a transaction and retrieving immutable data from a modular blockchain environment.

EX.NO: 05	CERTIFICATE GENERATION USING DAPPS
DATE:08.03.25	

AIM:

To create a decentralized application (DApp) that generates and stores certificates securely on the Ethereum blockchain using Remix IDE and PowerShell, ensuring tamper-proof verification and data integrity.

DESCRIPTION:

This experiment demonstrates how to create a Decentralized Application (DApp) for issuing and managing certificates using Solidity on the Ethereum blockchain. By deploying a smart contract via Remix IDE, an admin can securely generate certificates with unique details. The process includes writing, compiling, deploying, and interacting with a smart contract. This setup ensures transparency, immutability, and tamper-proof recordkeeping of credentials on the blockchain.

PROCEDURE:

Step 1: Open Remix IDE in your browser and create a new Solidity file named Certificate.sol in the file explorer panel.This will be used to write and compile your smart contract for the certificate system.

Step 2 : Write the smart contract by copying and pasting the Solidity code into Certificate.sol.This code defines the Certificate struct, an admin-only issue function, and a mapping to store issued certificates by ID.

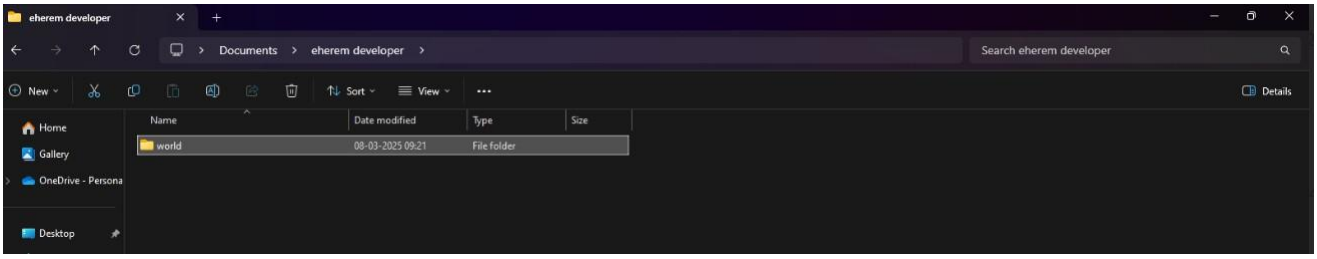
Step 3: Navigate to the “Solidity Compiler” tab, select version 0.8.20 (as defined in the pragma line), and click “Compile Certificate.sol”.Ensure there are no errors in the code to proceed to deployment.

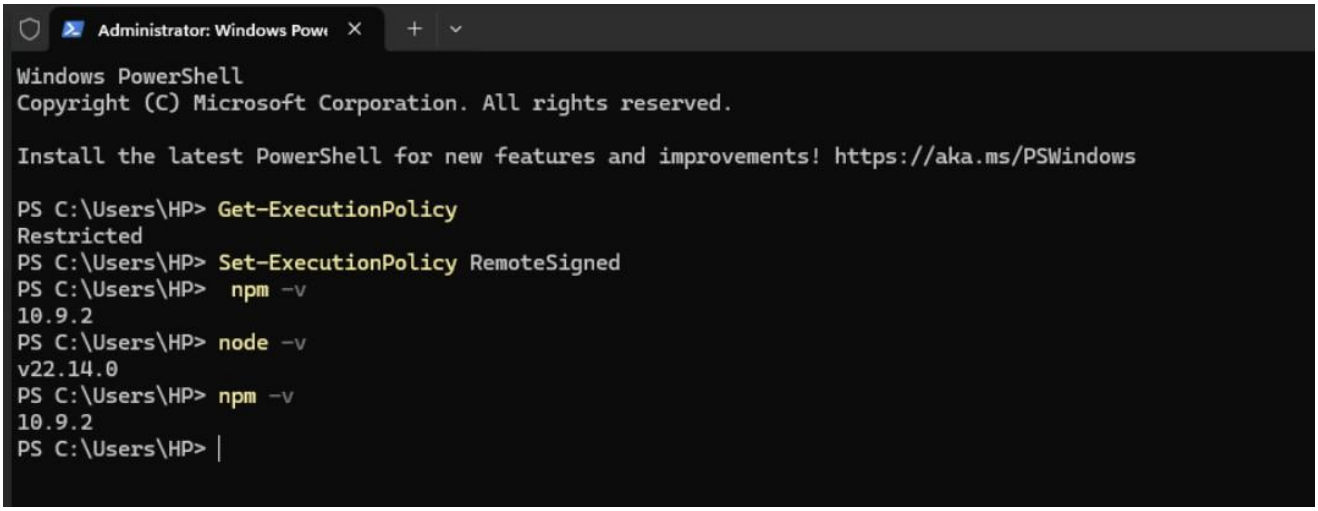
Step 4: Go to the “Deploy & Run Transactions” tab, choose either “Remix VM (London)” for local testing or “Injected Provider - MetaMask” for using the Ethereum network, and click “Deploy”.Confirm the transaction in MetaMask if using an external wallet.

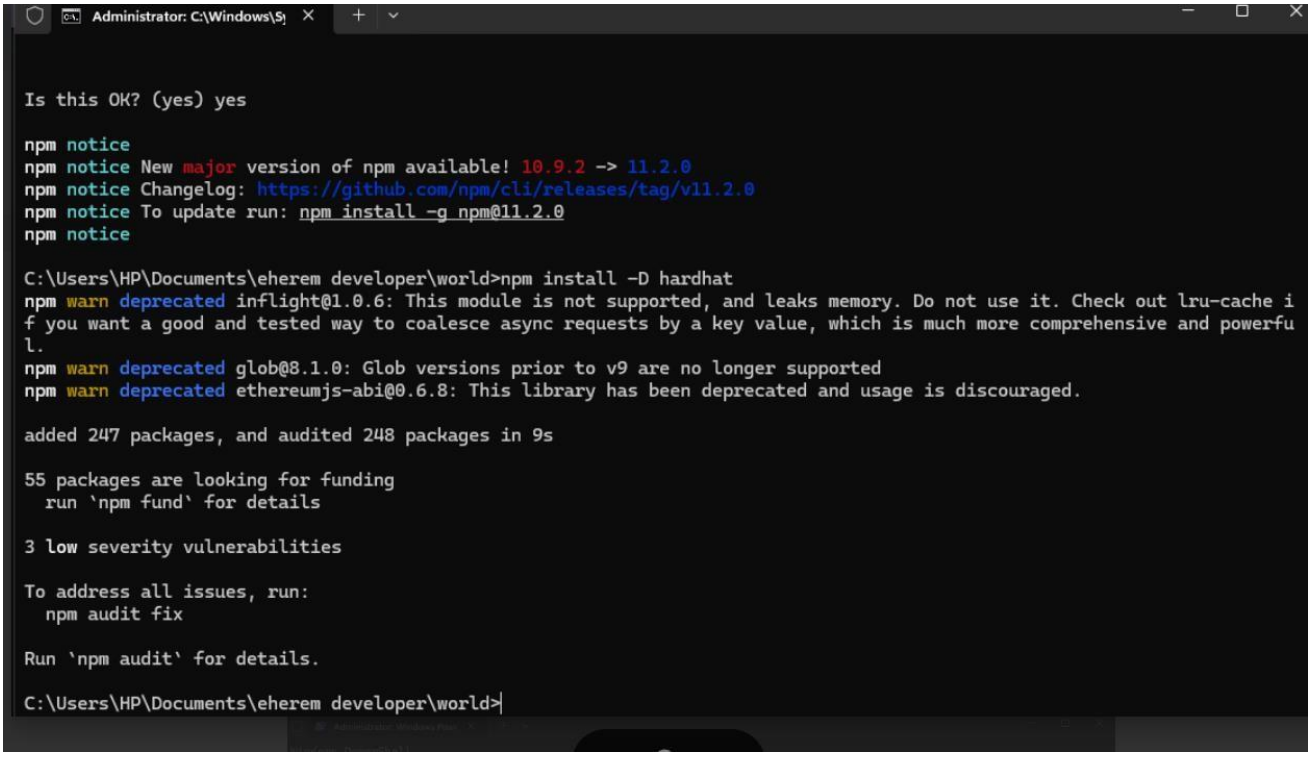
Step 5: After deployment, expand the contract under “Deployed Contracts” and locate the issue function.Fill in the required details (e.g., 1, "John Doe", "Blockchain Development", "A", "01-04-2025") and click “Transact” to issue a certificate.

Step 6: To verify the certificate details, enter the same certificate ID in the Certificates function input box and click to retrieve the stored certificate information.This will display the student’s name, course, grade, and date associated with the ID.

OUTPUT:

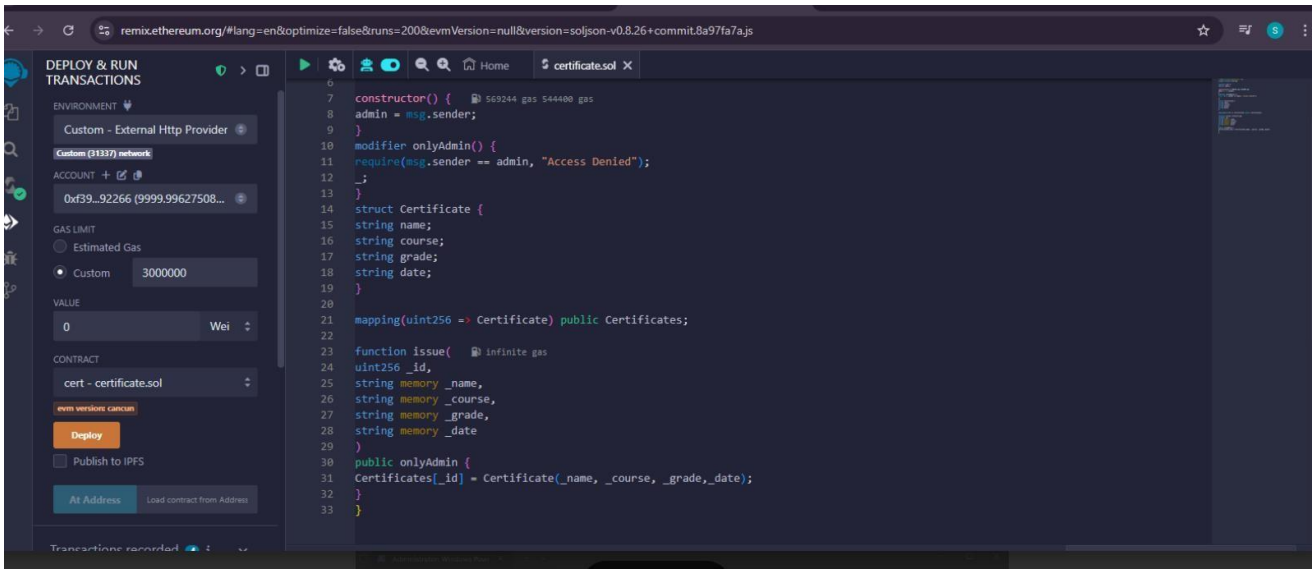

eherem developer
Documents
eherem developer
Search eherem developer
New
Sort
View
Details
Home
Gallery
OneDrive - Persona
Desktop
Name
Date modified
Type
Size
world
08-03-2025 09:21
File folder

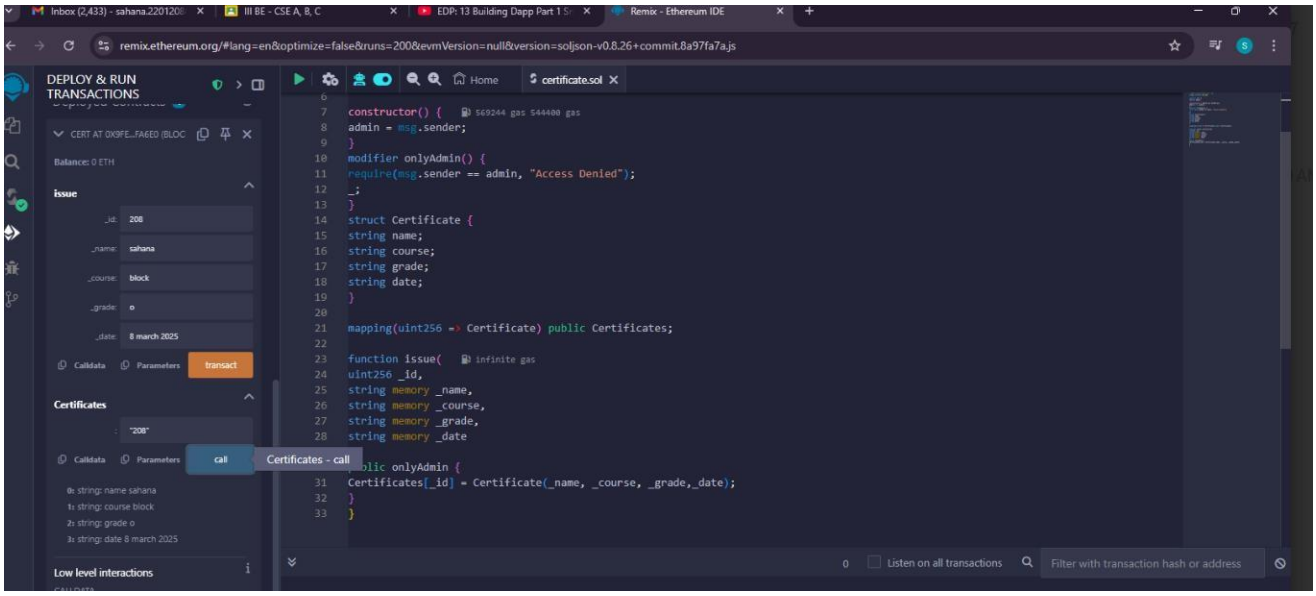

Administrator: Windows Powe
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\HP> Get-ExecutionPolicy
Restricted
PS C:\Users\HP> Set-ExecutionPolicy RemoteSigned
PS C:\Users\HP> npm -v
10.9.2
PS C:\Users\HP> node -v
v22.14.0
PS C:\Users\HP> npm -v
10.9.2
PS C:\Users\HP>

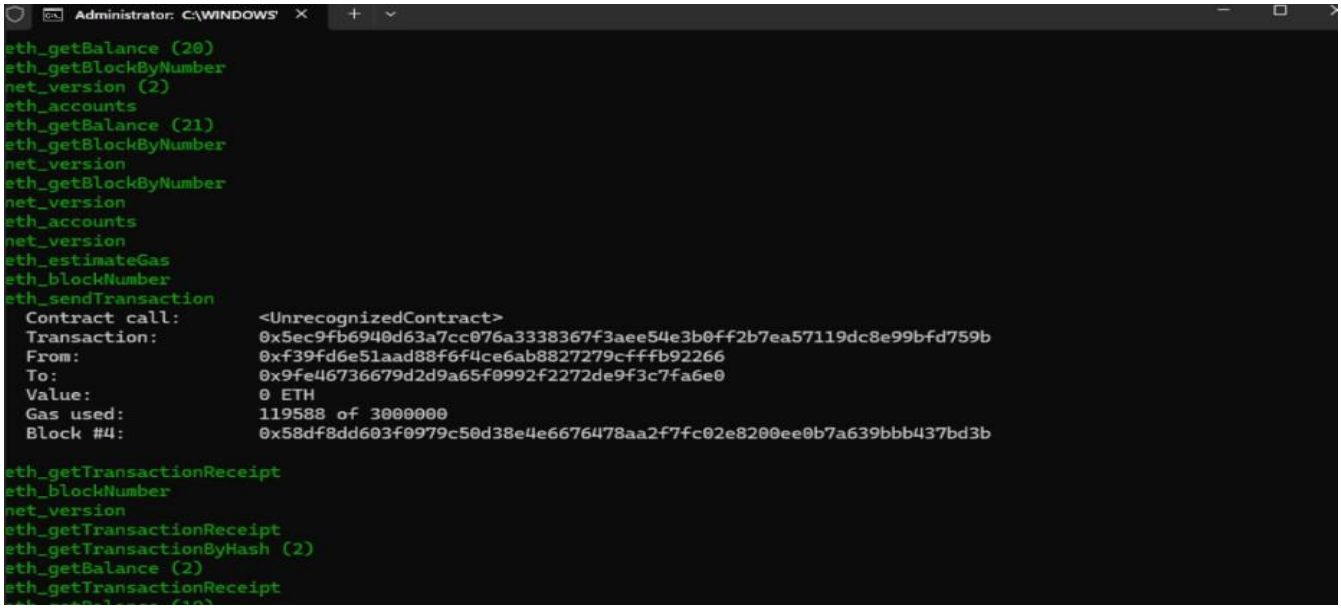

Administrator: C:\Windows\S
Is this OK? (yes) yes
npm notice
npm notice New major version of npm available! 10.9.2 -> 11.2.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v11.2.0
npm notice To update run: npm install -g npm@11.2.0
npm notice
C:\Users\HP\Documents\eherem developer\world>npm install -D hardhat
npm warn deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerful.
npm warn deprecated glob@8.1.0: Glob versions prior to v9 are no longer supported
npm warn deprecated ethereumjs-abi@0.6.8: This library has been deprecated and usage is discouraged.
added 247 packages, and audited 248 packages in 9s
55 packages are looking for funding
run `npm fund` for details
3 low severity vulnerabilities
To address all issues, run:
npm audit fix
Run `npm audit` for details.
C:\Users\HP\Documents\eherem developer\world>

```
Administrator: C:\WINDOWS'

added 247 packages, and audited 248 packages in 9s

55 packages are looking for funding
  run `npm fund` for details

3 low severity vulnerabilities

To address all issues, run:
  npm audit fix

Run `npm audit` for details.

C:\Users\HP\Documents\eherem developer\world>npx hardhat init
888    888                      888 888               888
888    888                      888 888               888
888    888                      888 888               888
8888888888  8888b.  888d888 .d88888 88888b.   8888b.  888888
888    888     "88b 888P"  d88" 888 888 "88b     "88b 888
888    888 .d888888 888    888  888 888  888 .d888888 888
888    888 888  888 888    Y88b 888 888  888 888  888 Y88b.
888    888 "Y888888 888     "Y88888 888  888 "Y888888  "Y888

Welcome to Hardhat v2.22.19

? What do you want to do? ...
> Create a JavaScript project
  Create a TypeScript project
  Create a TypeScript project (with Viem)
  Create an empty hardhat.config.js
  Quit
```

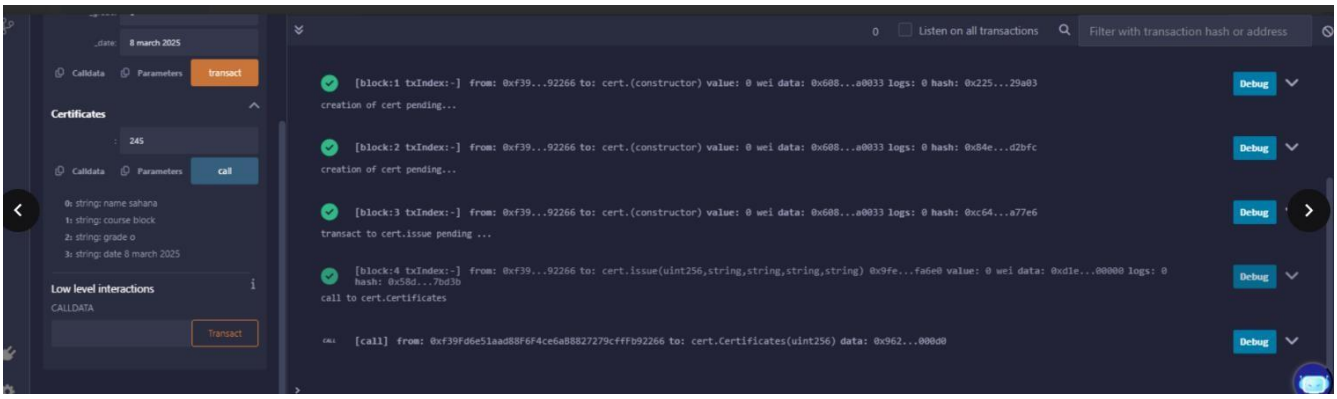
```
Administrator: C:\WINDOWS'

C:\Users\HP\Documents\eherem developer\world>npx hardhat node
√ Help us improve Hardhat with anonymous crash reports & basic usage data? (Y/n) · y
Started HTTP and WebSocket JSON-RPC server at http://127.0.0.1:8545/

Accounts
========

WARNING: These accounts, and their private keys, are publicly known.
Any funds sent to them on Mainnet or any other live network WILL BE LOST.

Account #0: 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 (10000 ETH)
Private Key: 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80

Account #1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (10000 ETH)
Private Key: 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d

Account #2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (10000 ETH)
Private Key: 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a

Account #3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (10000 ETH)
Private Key: 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

Account #4: 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (10000 ETH)
Private Key: 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a

Account #5: 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (10000 ETH)
Private Key: 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba

Account #6: 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (10000 ETH)
```


remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.26+commit.8a97fa7a.js
DEPLOY & RUN TRANSACTIONS
ENVIRONMENT
Custom - External Http Provider
Custom (31337) network
ACCOUNT
0xf39...92266 (9999.99627508...
GAS LIMIT
Estimated Gas
Custom
3000000
VALUE
0
Wei
CONTRACT
cert - certificate.sol
evm version: cancun
Deploy
Publish to IPFS
At Address
Load contract from Address
Home
certificate.sol
constructor() { 569244 gas 544400 gas
admin = msg.sender;
}
modifier onlyAdmin() {
require(msg.sender == admin, "Access Denied");
_;
}
struct Certificate {
string name;
string course;
string grade;
string date;
}
mapping(uint256 => Certificate) public Certificates;
function issue(infinite gas
uint256 _id,
string memory _name,
string memory _course,
string memory _grade,
string memory _date
)
public onlyAdmin {
Certificates[_id] = Certificate(_name, _course, _grade,_date);
}
}


Inbox (2,433) - sahana.2201208
III BE - CSE A, B, C
EDP: 13 Building Dapp Part 1 Sr
Remix - Ethereum IDE
remix.ethereum.org/#lang=en&optimize=false&runs=200&evmVersion=null&version=soljson-v0.8.26+commit.8a97fa7a.js
DEPLOY & RUN TRANSACTIONS
CERT AT 0X9FE...FA6ED (BLOC
Balance: 0 ETH
Issue
_id: 208
_name: sahana
_course: block
_grade: o
_date: 8 march 2025
Calldata
Parameters
transact
Certificates
"208"
Calldata
Parameters
call
Certificates - call
0: string: name sahana
1: string: course block
2: string: grade o
3: string: date 8 march 2025
Low level interactions
Home
certificate.sol
constructor() { 569244 gas 544400 gas
admin = msg.sender;
}
modifier onlyAdmin() {
require(msg.sender == admin, "Access Denied");
_;
}
struct Certificate {
string name;
string course;
string grade;
string date;
}
mapping(uint256 => Certificate) public Certificates;
function issue(infinite gas
uint256 _id,
string memory _name,
string memory _course,
string memory _grade,
string memory _date
public onlyAdmin {
Certificates[_id] = Certificate(_name, _course, _grade,_date);
}
}
0
Listen on all transactions
Filter with transaction hash or address


Administrator: C:\WINDOWS
eth_getBalance (20)
eth_getBlockByNumber
net_version (2)
eth_accounts
eth_getBalance (21)
eth_getBlockByNumber
net_version
eth_getBlockByNumber
net_version
eth_accounts
net_version
eth_estimateGas
eth_blockNumber
eth_sendTransaction
Contract call: <UnrecognizedContract>
Transaction: 0x5ec9fb6940d63a7cc076a3338367f3aee54e3b0ff2b7ea57119dc8e99bfd759b
From: 0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
To: 0x9fe46736679d2d9a65f0992f2272de9f3c7fa6e0
Value: 0 ETH
Gas used: 119588 of 3000000
Block #4: 0x58df8dd603f0979c50d38e4e6676478aa2f7fc02e8200ee0b7a639bbb437bd3b
eth_getTransactionReceipt
eth_blockNumber
net_version
eth_getTransactionReceipt
eth_getTransactionByHash (2)
eth_getBalance (2)
eth_getTransactionReceipt

```
Administrator: C:\Windows\S
Microsoft Windows [Version 10.0.22631.4974]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Documents\eherem developer\world>npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (world)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author:
license: (ISC)
About to write to C:\Users\HP\Documents\eherem developer\world\package.json:

{
  "name": "world",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
```




```
Administrator: C:\WINDOWS
eth_getBalance (2)
eth_getTransactionReceipt
eth_getBalance (19)
net_version
eth_accounts
eth_getBalance (21)
eth_getBlockByNumber
net_version
eth_getBlockByNumber
net_version
eth_call
  Contract call:       <UnrecognizedContract>
  From:                0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266
  To:                  0x9fe46736679d2d9a65f0992f2272de9f3c7fa6e0

eth_getCode
net_version
eth_accounts
eth_getBalance (21)
```

RESULT:

Thus, successfully explored and analyze the CoinMarketCap website, understood Bitcoin's market data, historical trends, and various analytical tools and features available for user interaction and research.

EX.NO: 06	CLINICAL TRAIL USING DAPPS
DATE:13.03.25	

AIM:

To develop a decentralized application (DApp) that manages participant enrollment and trial data recording for clinical trials using blockchain technology, ensuring data transparency, security, and authenticity.

DESCRIPTION:

This experiment demonstrates how decentralized applications (DApps) can be used to manage clinical trial processes securely. Using Solidity and Remix IDE, we develop a smart contract to handle participant enrollment and store trial data immutably. The contract ensures only authorized actions can be performed by the admin, with each interaction securely logged on the blockchain. Integration with MetaMask and Ganache simulates a local Ethereum environment for deployment and testing. This approach ensures data transparency, prevents tampering, and allows reliable retrieval of clinical records.

PROCEDURE:

Step 1: Set up the environment by installing the MetaMask browser extension and running Ganache locally.Copy the RPC server URL from Ganache (e.g., http://127.0.0.1:7545) and import one of Ganache's private keys into MetaMask to simulate an Ethereum wallet on the local network.

Step 2 : Open Remix IDE in your browser and create a new file named ClinicalTrial.sol.Paste the provided smart contract code into the editor to define participant enrollment, data recording, and retrieval functions.

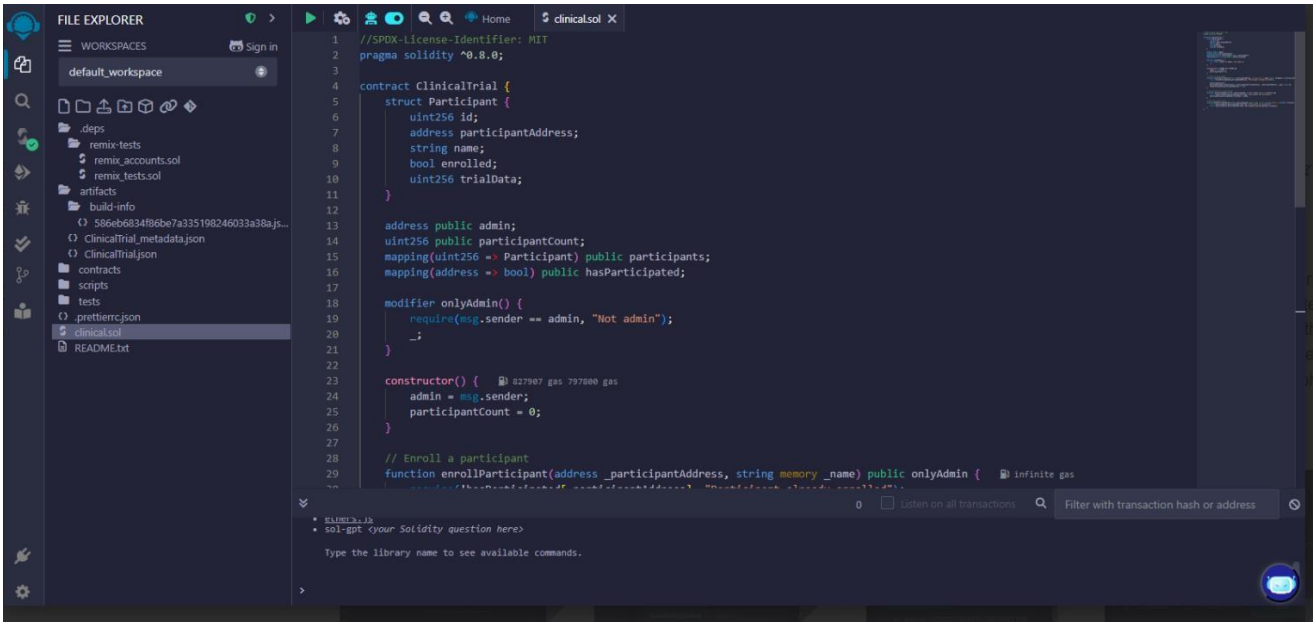
Step 3: Navigate to the Solidity Compiler tab in Remix and select compiler version ^0.8.0.Click "Compile ClinicalTrial.sol" and ensure that the compilation completes without errors.

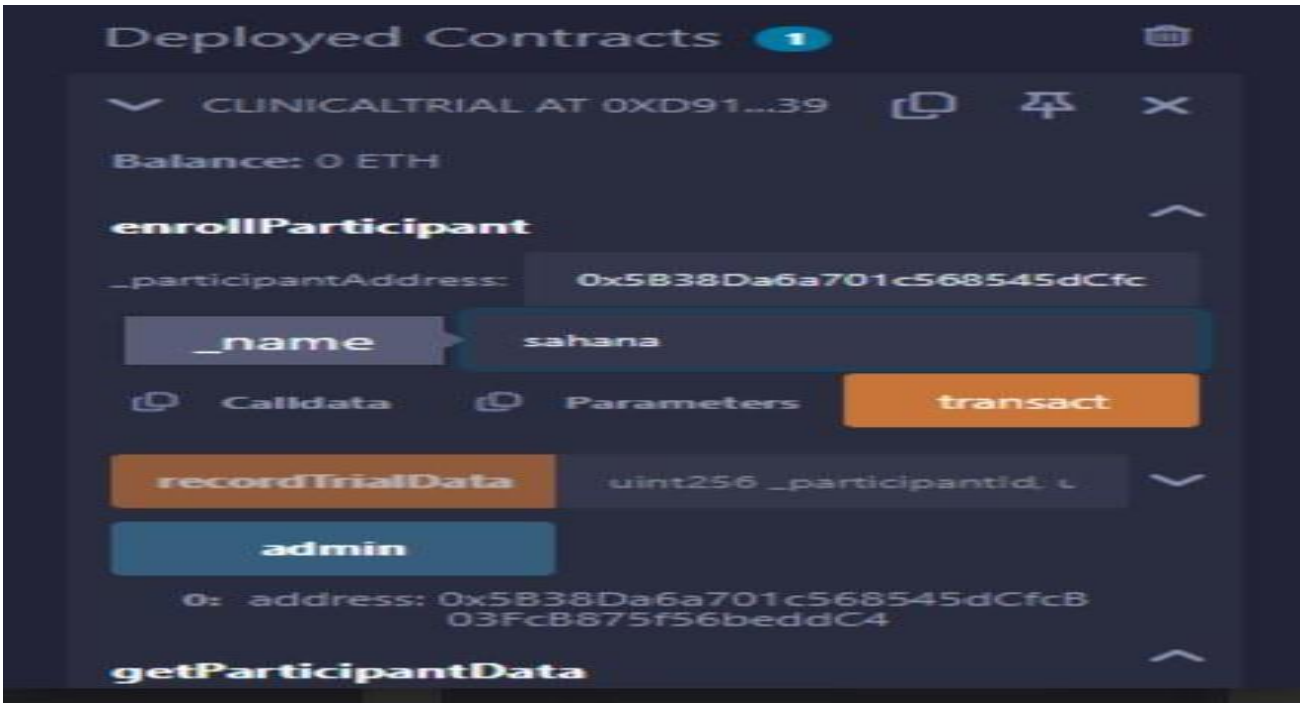
Step 4: In the "Deploy & Run Transactions" panel, select "Injected Web3" as the environment to connect Remix with MetaMask and Ganache.Click "Deploy", confirm the transaction in MetaMask, and wait for the smart contract to be deployed.

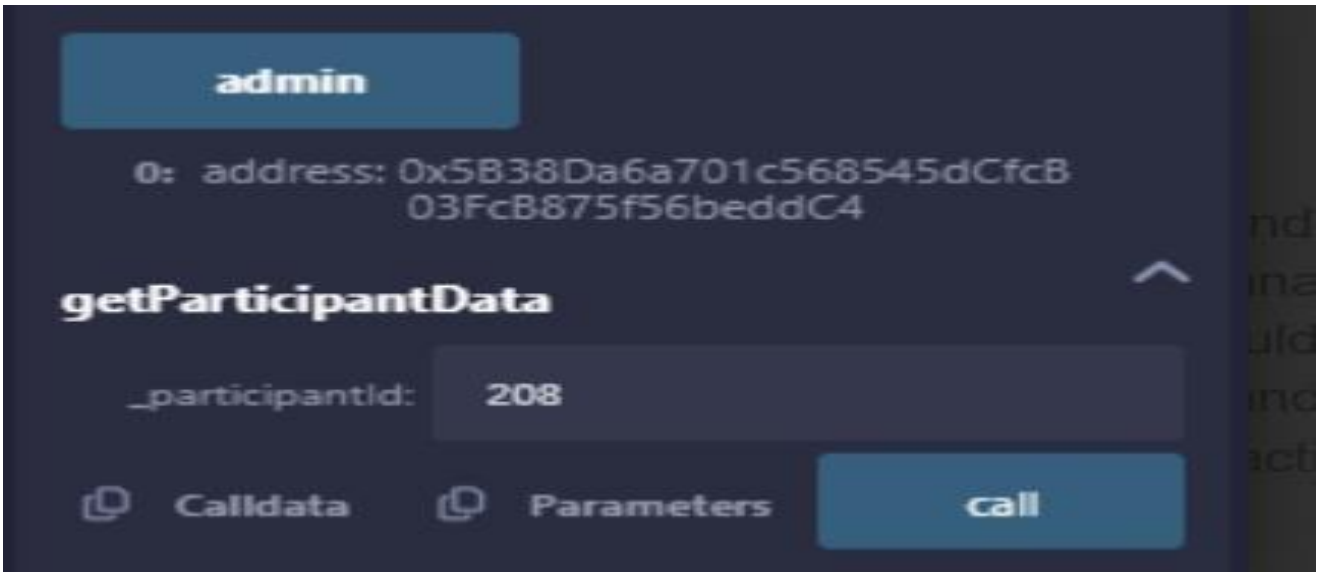
Step 5: After deployment, use the enrollParticipant() function to add new trial participants by entering their Ethereum address and name.Next, call recordTrialData() with the participant ID and trial data value. To verify data, use getParticipantData() by providing the participant ID.

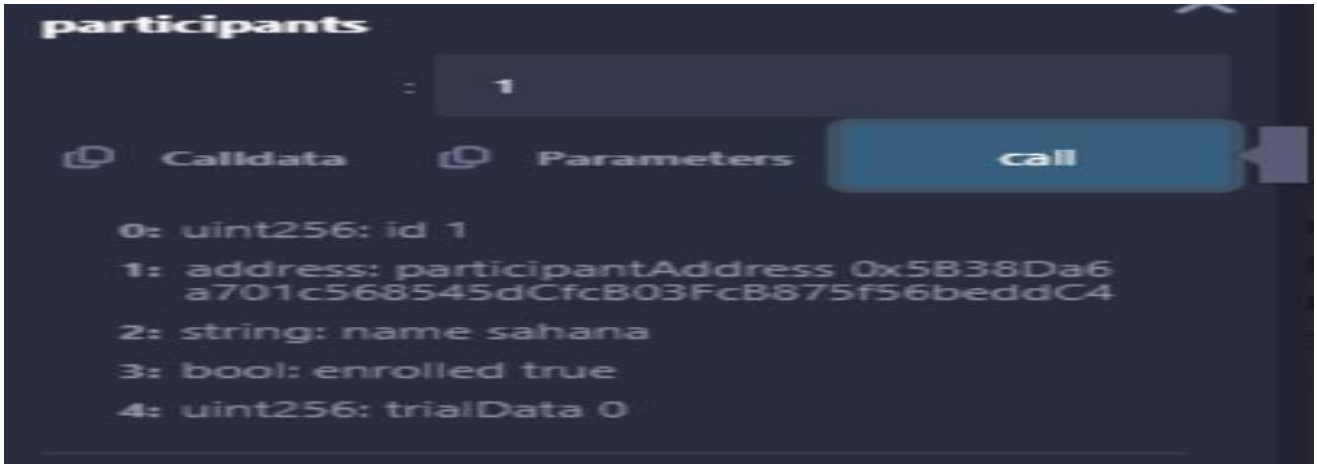
Step 6: Keep Ganache running in the background and monitor real-time transaction logs in PowerShell or Ganache UI.Each contract interaction—enrollment, data recording, or retrieval—will reflect as a transaction on the blockchain, ensuring secure and traceable data handling.

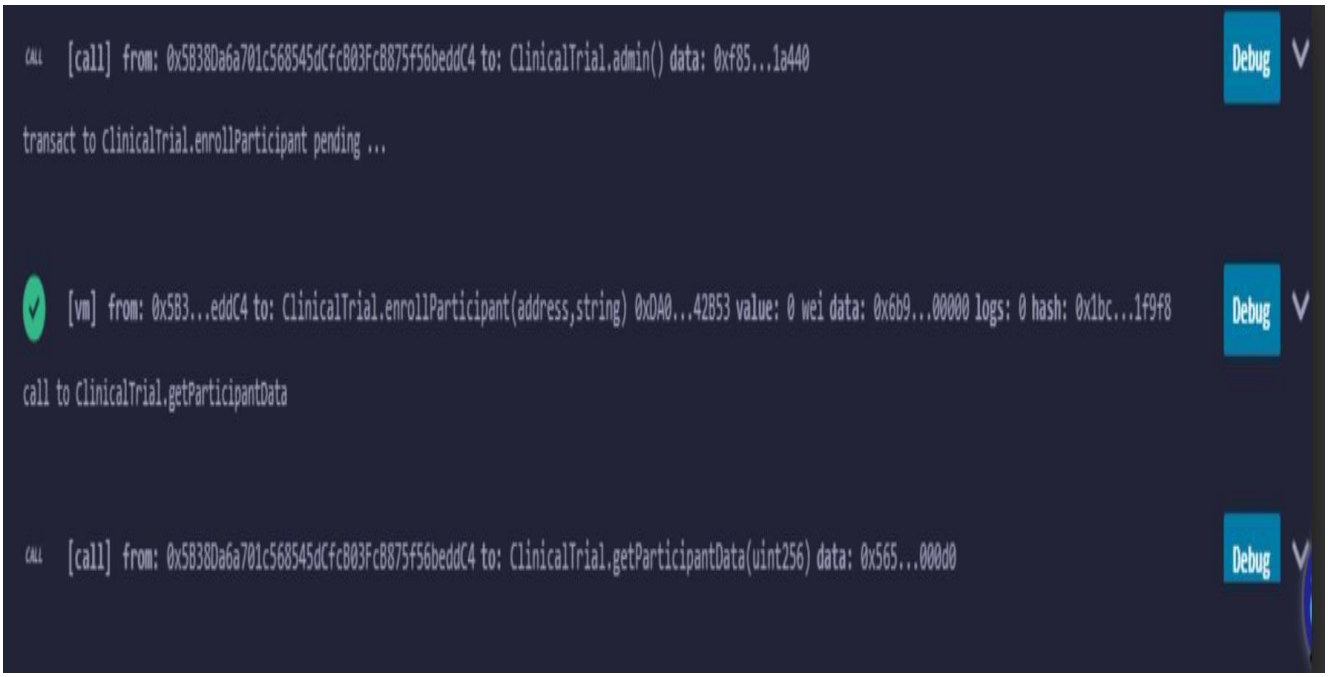
OUTPUT:




Deployed Contracts 1
CLINICALTRIAL AT 0XD91...39
Balance: 0 ETH
enrollParticipant
_participantAddress: 0x5B38Da6a701c568545dCfc
_name
sahana
Calldata
Parameters
transact
recordTrialData
uint256 _participantId,
admin
0: address: 0x5B38Da6a701c568545dCfcB 03FcB875f56beddC4
getParticipantData


admin
0: address: 0x5B38Da6a701c568545dCfcB 03FcB875f56beddC4
getParticipantData
_participantId: 208
Calldata
Parameters
call


participants
: 1
Calldata
Parameters
call
0: uint256: id 1
1: address: participantAddress 0x5B38Da6 a701c568545dCfcB03FcB875f56beddC4
2: string: name sahana
3: bool: enrolled true
4: uint256: trialData 0



RESULT:

Thus, successfully explored and analyze the CoinMarketCap website, understood Bitcoin's market data, historical trends, and various analytical tools and features available for user interaction and research.

EX.NO: 07	LAND REGISTRATION USING ETHEREUM
DATE:20.03.25	

AIM:

Develop a decentralized land registration system on the Ethereum blockchain. Securely register and store land ownership records on the blockchain.Ensure transparency, immutability, and security of land information.

DESCRIPTION:

This project aims to create a decentralized Land Registration DApp using Ethereum blockchain technology. It leverages Solidity for writing smart contracts and uses tools like Remix IDE, MetaMask, and Ganache to deploy and test the application. By storing land records on the blockchain, the system ensures transparency, immutability, and tamper-proof record management for property ownership.

PROCEDURE:

1. Designing the Smart Contract:

- Define the data structure for land records (e.g., owner name, address, plot number, registration date).
- Write a Solidity smart contract for land registration, ownership transfer, and querying records.
- Compile the smart contract using Remix or Truffle.

2. Implementing the Frontend Interface:

- Develop a user interface using HTML, CSS, and JavaScript.
- Include forms to enter land details, transfer ownership, and query records.
- Employ web3.js or ethers.js to bridge the frontend and smart contract.

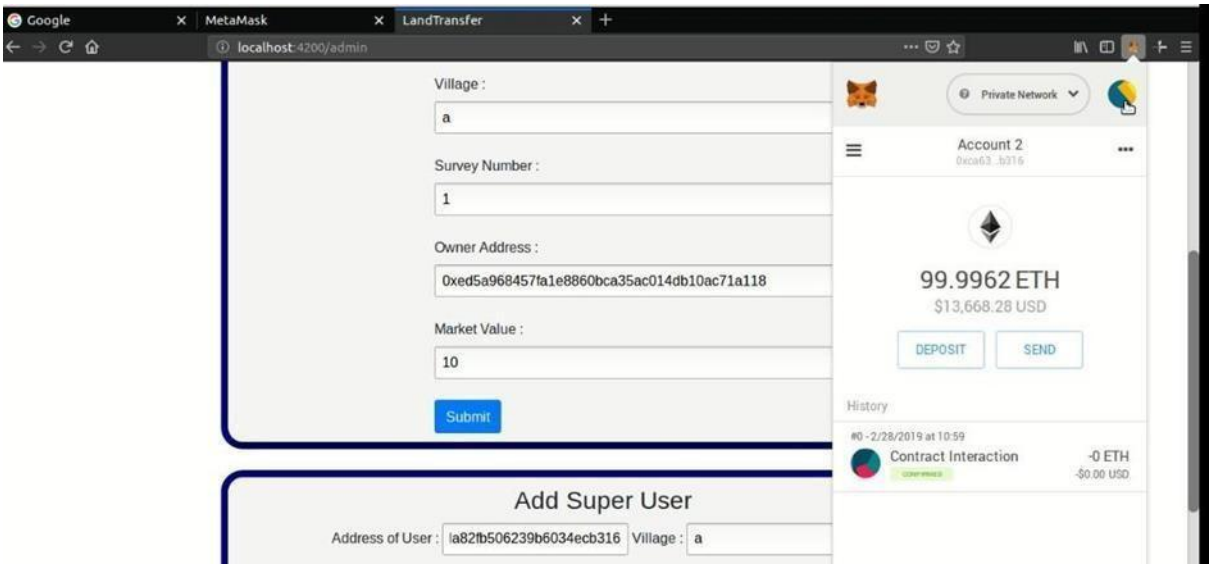
3. Deploying the Smart Contract to Ethereum:

- Employ Ganache for local deployment or deploy to Ethereum testnet/mainnet.
- Provide adequate Ether for deployment and gas costs.
- Deploy the contract via Remix or Truffle.

4. Testing the Application:

- Connect Metamask to the respective Ethereum network.
- Open the frontend in a browser and test all functionality.
- Check that land records are stored immutably and securely on the blockchain.

OUTPUT:



RESULT:

Effectively developed and implemented a decentralized land registration system on the Ethereum blockchain. The system enables secure, tamper-proof, and transparent land record management. Emphasized the capabilities of blockchain technology in substituting conventional land registration systems with a secure and auditable digital alternative.

EX.NO: 08	SMART TOLL MANAGEMENT SYSTEM
DATE:27.03.25	

AIM:

To develop a decentralized Smart Toll Management System using Ethereum blockchain technology, enabling secure, automated toll fee collection and transaction tracking through smart contracts.

DESCRIPTION:

This project implements a blockchain-based toll management system that eliminates manual toll operations and ensures transparency. The system uses a Solidity smart contract to register vehicles, collect toll charges, and record transaction history. It features a user-friendly frontend built with HTML, CSS, and JavaScript, which interacts with the Ethereum network through web3.js or ethers.js. The smart contract is deployed on a blockchain network using Remix IDE or Truffle, with testing conducted on Ganache and interaction managed via MetaMask.

PROCEDURE:

1. **Designing the Smart Contract:**

   - Define the data structure for toll booth transactions, including fields such as vehicle ID, toll amount, timestamp, etc.
   - Write a Solidity smart contract to implement the toll management logic, including functions for registering vehicles, deducting toll charges, and querying transaction history.
   - Compile the smart contract using a Solidity compiler such as Remix or Truffle.

2. **Incorporating the Frontend Interface:**

   - Design a frontend interface using HTML, CSS, and JavaScript to enable interaction between vehicle users and the toll management system.
   - Develop a clean and intuitive user interface offering vehicle registration, payment of tolls, and viewing transaction history through forms.
   - Interact with the Ethereum blockchain by using web3.js or ethers.js for interacting with the already deployed smart contract.

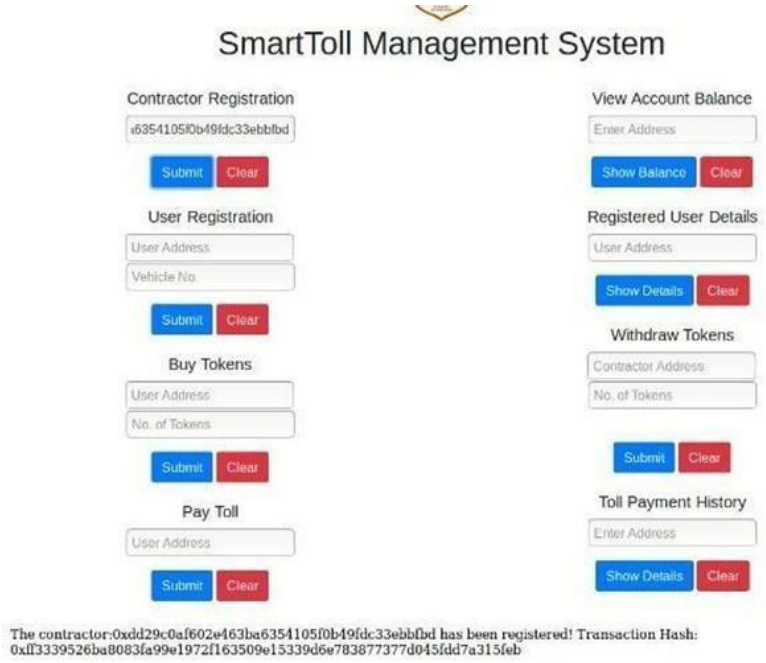
3. Deploying the Smart Contract to Ethereum:

   - If deploying with Ganache, deploy the smart contract to the local test and development blockchain network. If deploying to the Ethereum mainnet or testnet, make sure you have enough Ether to pay for the gas costs.
   - Deploy the compiled smart contract to the target Ethereum network using a deployment tool like Truffle or Remix.

4. Testing the Application:

- Link Metamask to the Ethereum network where the deployed smart contract is located.
- Launch the frontend interface in a web browser and confirm that it properly interacts with the deployed smart contract.
- Test the toll management feature by registering vehicles, driving through toll booths, and checking toll charges debited from vehicle accounts.
- Confirm that toll transactions are stored immutably on the Ethereum blockchain and can be viewed by anyone who has the right permissions.

OUTPUT:



RESULT:

Thus, a decentralized Smart Toll Management System was successfully developed and tested. The system enabled vehicle registration, toll collection, and transaction tracking in a transparent and tamper-proof manner on the Ethereum blockchain, ensuring automated and secure toll operations.

EX.NO: 09	CONNECTING METAMASK, GANACHE, AND ETHEREUM
DATE:03.04.25	

AIM:

To deploy a smart contract to a local Ethereum blockchain using Ganache, configure MetaMask to interact with the network, and perform transactions using a connected front-end or browser console.

DESCRIPTION:

This experiment focuses on setting up a local blockchain development environment and interacting with a deployed smart contract. Ganache is used to simulate an Ethereum network locally, providing test accounts and Ether for development. MetaMask is configured to connect to the Ganache network, enabling secure interaction with the blockchain. A basic Solidity smart contract is compiled and deployed using Remix IDE or Truffle, and the deployed contract is accessed via its address and ABI. Developers can then interact with the smart contract by calling functions and performing blockchain transactions in a safe, local environment..

PROCEDURE:

1. Deploying a Smart Contract to Ganache:

- Open the Ganache app on your computer.
- Make sure Ganache is running and showing a list of accounts with private keys.
- If necessary, create a new workspace in Ganache.
- Create a basic smart contract in Solidity or use an existing one.
- Compile the smart contract with a Solidity compiler.
- Deploy the compiled smart contract to the Ganache network by connecting to it using a development tool like Truffle or Remix.

2. Configuring Metamask for Ethereum:

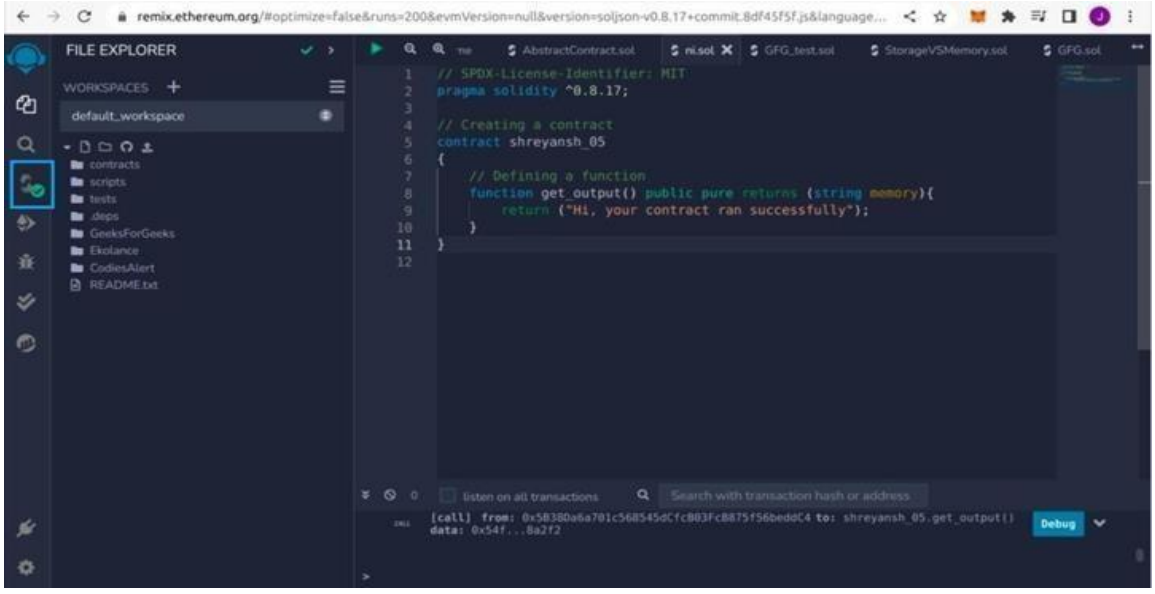
- Open your web browser and make sure the Metamask extension is installed.
- Click on the Metamask icon from the browser toolbar to launch the extension.

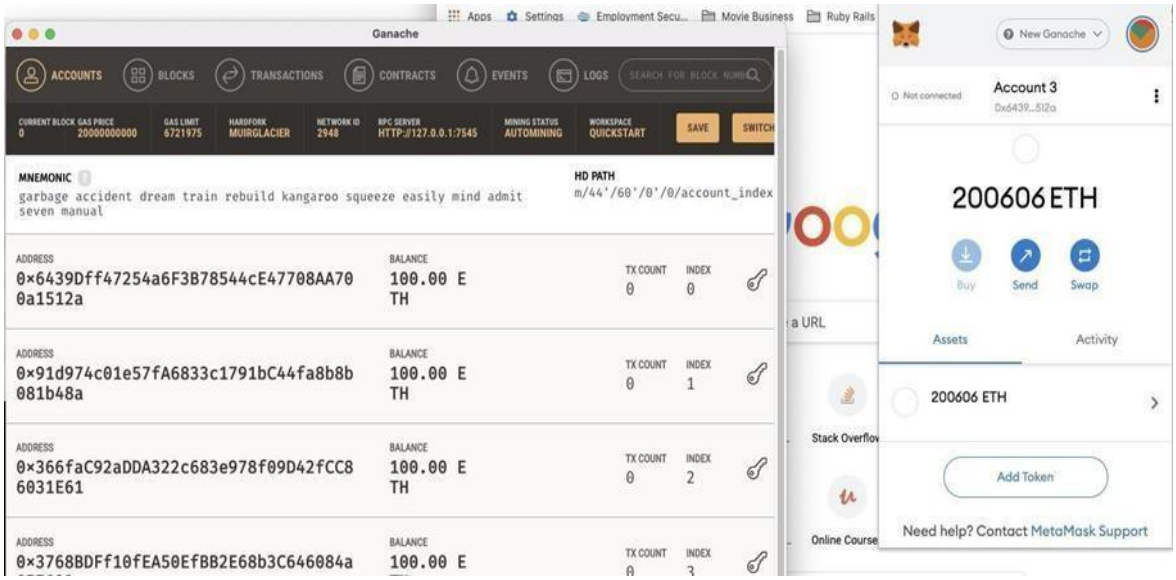
- If you have not done so already, create a new Ethereum wallet or import an existing wallet into Metamask.
- In Metamask, click on the network dropdown menu and choose "Custom RPC. Paste the RPC server endpoint supplied by Ganache into the "New RPC URL" field.
- Click "Save" to link Metamask to the Ganache network.

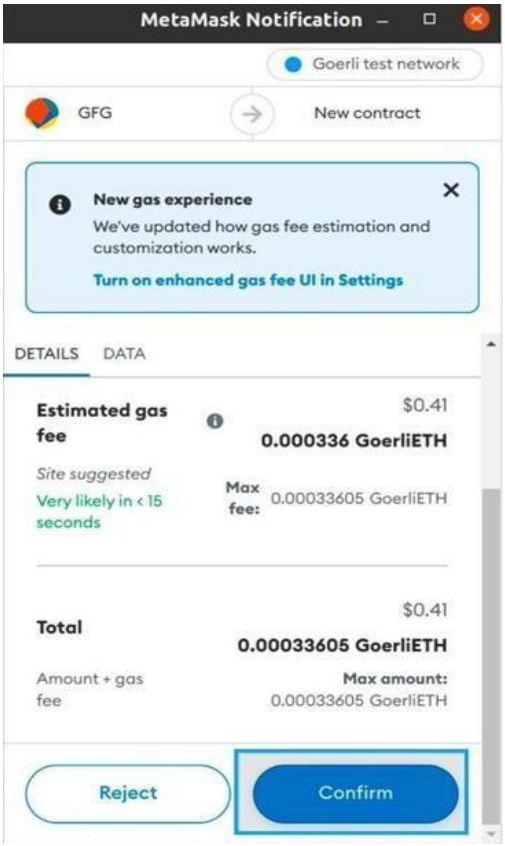
3. Interacting with the Smart Contract:

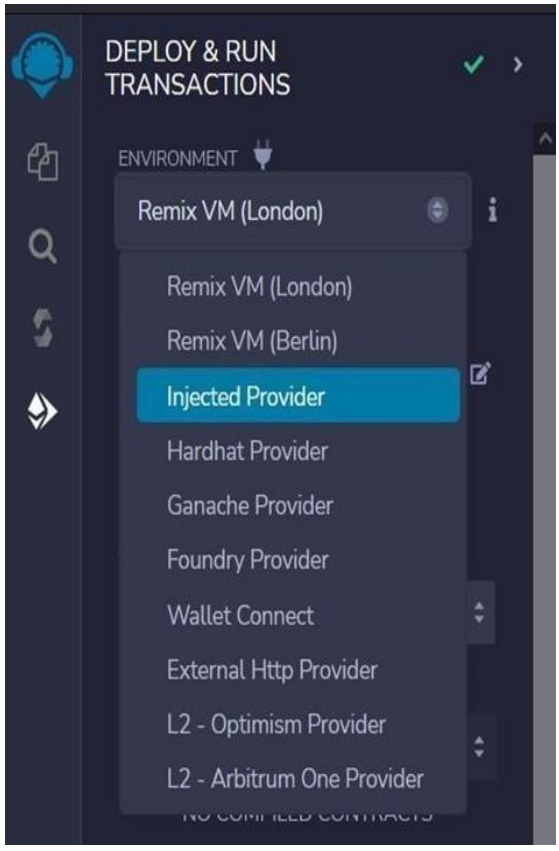
- Once Metamask is linked to the Ganache network, switch to the account used to deploy the smart contract.
- Copy the address of the deployed smart contract from Ganache.
- In your development environment or browser console, create a contract object using the address of the smart contract and ABI (Application Binary Interface).
- Use the contract object to interact with the deployed smart contract, for example, by calling functions or sending transactions.

OUTPUT:









RESULT:

Thus, the smart contract was successfully deployed on the Ganache local blockchain, and MetaMask was configured to interact with it. Transactions were executed successfully through a contract interface, demonstrating the process of interacting with Ethereum smart contracts in a controlled development setup.